

PROYECTO FINAL - OPTIMIZACIÓN

Presentado a:

Henry Armando Percy Guzmán

Presentado por:

Lina Sofía Amaya Rodríguez
Laura Valentina Gonzalez Rincon
Camilo Ferney Londoño Moreno

Universidad Nacional de Colombia, sede Bogotá
Departamento de Sistemas e Industrial, Facultad de Ingeniería
Optimización
16 de junio de 2026

PROBLEMA 3 – 0/1 KNAPSACK PROBLEM (50 ÍTEMS).....	3
Revisión de Literatura.....	3
Referencias.....	4
Desarrollo del Ejercicio.....	5
1. Introducción.....	5
2. Fuente Académica.....	5
3. Formulación matemática.....	6
4. Restricciones.....	7
5. Solución por Gurobi.....	8
6. Análisis de Resultados.....	9
7. Conclusiones.....	10
PROBLEMA 7 – SET COVERING PROBLEM (50 FILAS – 200 COLUMNAS).....	11
Revisión de Literatura.....	11
Referencias.....	12
Desarrollo del Ejercicio.....	13
1. Introducción.....	13
2. Fuente Académica.....	13
3. Formulación matemática.....	14
4. Restricciones.....	14
5. Solución por Gurobi.....	15
6. Análisis de Resultados.....	16
7. Conclusiones.....	19
MODELO DE REDES.....	20
Revisión de Literatura.....	20
Desarrollo del Ejercicio.....	21
1. Introducción.....	21
2. Problemática.....	21
3. Descripción de la red.....	21
4. Características Descriptivas.....	22
5. Referencias.....	24

PROBLEMA 3 – 0/1 KNAPSACK PROBLEM (50 ÍTEMS)

Revisión de Literatura

El problema KNAPSACK o problema de la “mochila” es uno de los modelos más estudiados en la optimización combinatoria y la investigación de operaciones. Este modelo consiste en seleccionar un subconjunto de elementos con el objetivo de maximizar el beneficio total sin exceder una capacidad de peso determinada (W). En su variante 0/1, cada elemento puede ser seleccionado como máximo una vez; es decir, la variable de decisión asociada a cada elemento es binaria, a diferencia de otras variantes que permiten selecciones múltiples o fraccionarias (Cacchiani et al., 2022).

El problema de la mochila 0/1 (0/1 KP) está clasificado como NP-difícil en sentido débil (weakly NP-hard), lo que implica que a pesar de su complejidad teórica, puede resolverse en tiempo pseudo-polinomial mediante programación dinámica con complejidad ($O(nW)$) (Martello et al., 1999; Jookan et al., 2022). Esta característica lo distingue de los problemas fuertemente NP-difíciles y explica por qué, en la práctica, muchas instancias pueden resolverse eficientemente mediante algoritmos exactos especializados, como los algoritmos de programación dinámica mejorados propuestos por Martello et al. (1999), el algoritmo *expknap* (Pisinger, 1995) y el algoritmo *minknap* (Pisinger, 1997).

El 0/1 KP ocupa un lugar central en la investigación en optimización combinatoria. Las obras de síntesis y sistematización de Martello et al. (1999) y Kellerer et al. (2004) consolidaron una línea de investigación ya establecida y contribuyeron a su desarrollo como un área productiva dentro de la optimización combinatoria (Cacchiani et al., 2022). Asimismo, el problema es utilizado frecuentemente como referencia (benchmark) para evaluar y comparar métodos de solución en este campo (Kellerer et al., 2004). La amplitud de aplicaciones del 0/1 KP evidencia su relevancia práctica. Cacchiani et al. (2022) documentan que este problema aparece recurrentemente en sectores industriales como el transporte, la logística, el corte y empaque, las telecomunicaciones, la confiabilidad de sistemas, la publicidad, la inversión y la asignación presupuestaria.

En el desarrollo histórico del problema, destaca la contribución del profesor David Pisinger, investigador actualmente vinculado a la Universidad Técnica de Dinamarca (DTU) y reconocido por sus aportes al estudio de problemas de optimización combinatoria, diseño de redes, ruteo de vehículos y logística (Technical University of Denmark, s. f.).

Entre sus contribuciones al estudio del problema de la mochila sobresale el artículo *Where are the hard knapsack problems?* (Pisinger, 2005), en el que se analiza la dificultad de los conjuntos de instancias utilizados tradicionalmente para evaluar algoritmos exactos. El estudio identificó que muchas de estas instancias presentaban características estructurales que favorecen el desempeño de técnicas existentes,

particularmente aquellas basadas en estrategias de reducción de núcleo (core) desarrolladas en trabajos previos. Como resultado, se propusieron nuevas familias de instancias diseñadas para representar desafíos computacionales más exigentes. La influencia de este trabajo se mantiene vigente, como lo demuestra el estudio de Smith-Miles et al. (2021), quienes retomaron las ideas de Pisinger mediante la metodología de Instance Space Analysis para caracterizar sistemáticamente la dificultad de las instancias del problema de la mochila 0/1.

Referencias

● *Problema 3 – 0/1 Knapsack Problem*

Cacchiani, V., Iori, M., Locatelli, A., & Martello, S. (2022). Knapsack problems — An overview of recent advances. Part I: Single knapsack problems. *Computers & Operations Research*, 143, 105692. <https://doi.org/10.1016/j.cor.2021.105692>

Jookan, J., Leyman, P., & De Causmaecker, P. (2022). A new class of hard problem instances for the 0–1 knapsack problem. *European Journal of Operational Research*, 301(3), 841–854. <https://doi.org/10.1016/j.ejor.2021.12.009>.

Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack problems*. Springer.

Martello, S., Pisinger, D., & Toth, P. (1999). Dynamic Programming and Strong Bounds for the 0-1 Knapsack Problem. *Management Science*, 45(3), 414–424. <http://www.jstor.org/stable/2634886>

Pisinger, D. (1995). An expanding-core algorithm for the exact 0-1 knapsack problem. *European Journal of Operational Research*, 87(1), 175–187. [https://doi.org/10.1016/0377-2217\(94\)00013-3](https://doi.org/10.1016/0377-2217(94)00013-3).

Pisinger, D. (1997). A Minimal Algorithm for the 0-1 Knapsack Problem. *Operations Research*, 45(5), 758–767. <http://www.jstor.org/stable/172128>

Pisinger, D. (2005). Where are the hard knapsack problems?. *Computers & Operations Research*, 32(12), 2271–2284. <https://doi.org/10.1016/j.cor.2004.03.002>

Smith-Miles, K., Christiansen, J., Muñoz, M. A., & Baatar, D. (2021). Revisiting "Where are the hard knapsack problems?" via Instance Space Analysis. *Computers & Operations Research*, 128, 105184. <https://doi.org/10.1016/j.cor.2020.105184>

Technical University of Denmark. (s. f.). David Pisinger – DTU *Orbit Research Database*. Recuperado de <https://orbit.dtu.dk/en/persons/david-pisinger>

Desarrollo del Ejercicio

1. Introducción

El desarrollo de este apartado presenta el problema de la mochila (0/1) o 0/1 Knapsack Problem, utilizando una instancia de 50 ítems con pesos y beneficios no correlacionados, tal como fue presentado en el documento del proyecto final. Para obtener los datos concretos del ejercicio se acudió al enlace oficial <https://hjemmesider.diku.dk/~pisinger/codes.html>, el cual redirige a la página personal del profesor e investigador David Pisinger. La página está alojada en el dominio del Departamento de Ciencias de la computación de la Universidad de Copenhague (DIKU), siendo esta la fuente primaria reconocida en la literatura de optimización combinatoria para instancias de prueba del problema de la mochila.

De los tres paquetes de instancias disponibles en esta página (small coefficients, large coefficients, hard instances), se descargó el paquete de small coefficients, dentro del cual se identificó el archivo “*knapPI_1_50_1000_1*”. El nombre del archivo codifica las características requeridas: el primer dígito (1) indica la instancia no correlacionada, el segundo (50) indica la cantidad de ítems, el tercero (1000) indica el rango de coeficientes usado para generar pesos y beneficios y el cuarto (1) indica el ejercicio elegido. El archivo contiene: los datos de cada ítem, la capacidad de la mochila ($c = 995$) y el valor óptimo ya calculado por el profesor Pisinger ($z^* = 8373$). Los datos anteriores permiten verificar de forma independiente que la solución obtenida con Gurobi es correcta.

Para optimizar el trabajo y evitar la carga manual del archivo en cada ejecución del notebook, se decidió subir el archivo de datos (*Knapsack_50items.txt*) al repositorio del proyecto en GitHub: <https://github.com/CamiloLM/proyecto-optimizacion/tree/main/knapsack>, el código en Gurobi descarga automáticamente el archivo desde la URL del repositorio mediante la función `urlretrieve()` de la librería estandar `urllib.request`, verificando si el archivo existe localmente para no descargarlo de manera constante.

2. Fuente Académica

Referencia Principal: Pisinger, D. (2005). Where are the hard knapsack problems?. *Computers & Operations Research*, 32(12), 2271–2284. <https://doi.org/10.1016/j.cor.2004.03.002>

El problema de la mochila 0/1 es uno de los 21 problemas NP-completos clásicos y constituye un pilar en la literatura de optimización combinatoria, ya que su estructura simple pero computacionalmente difícil lo convierte en un caso de prueba estándar para algoritmos exactos (programación dinámica, ramificación y acotamiento) y métodos heurísticos. El trabajo del profesor Pisinger, del cual proviene el ejercicio analizado en este apartado, identifica que características hacen que ciertas instancias del problema sean más difíciles de resolver que otras, y propone el conjunto de instancias benchmark utilizado universalmente desde entonces por la comunidad académica.

3. Formulación matemática

El problema presentado se formula con un modelo de Programación Lineal Entera Binaria (PLEB). A continuación se presenta cada componente:

Se define $i = 1, 2, 3, \dots, 50$ como el índice de cada ítems disponible. El conjunto tiene cardinalidad de 50 porque así lo especifica la rúbrica.

Para cada ítem i se conocen dos valores fijos extraídos del archivo de datos: el beneficio p_i y el peso w_i . Adicionalmente, $c = 995$ representa la capacidad máxima de la mochila. Estos parámetros son constantes en el problema (no son variables de decisión), porque ya vienen determinados y no se pueden modificar durante la optimización.

Además, se define una **variable** binaria para cada ítem:

$$x_i \in \{0, 1\}, \quad i = 1, 2, 3, \dots, 50$$

donde $x_i = 1$ si el ítem se selecciona y $x_i = 0$ si el ítem no se selecciona. La justificación de usar una variable binaria no continua es central, ya que en la versión 0/1 del ejercicio de la mochila no se permite fraccionar ítems. Si se permite $x_i \in [0, 1]$ continuo el problema se convertiría en una mochila fraccionaria que se resolvería mediante un algoritmo greedy que ordena ítems por su ratio p_i/w_i . Es justamente la restricción de integralidad binaria (mencionada más adelante) la que convierte este problema en NP-difícil y justifica el uso de un solver de optimización entera como Gurobi.

La **función objetivo** del ejercicio quedaría de la siguiente manera:

$$\text{Maximizar } z = \sum_{i=1}^{50} p_i \times x_i$$

Se maximiza porque el objetivo del problema es obtener el mayor beneficio posible, no minimizar costos ni pesos. Cada término $p_i x_i$ contribuye al total solo si el ítem fue seleccionado, es decir, si $x_i = 0$ se anula automáticamente sin necesidad de condicionales adicionales en el modelo.

La restricción de capacidad quedaría de la siguiente forma:

$$\sum_{i=1}^{50} w_i \times x_i \leq 995$$

Esta restricción menciona la limitación física del ejercicio: la suma de los pesos de los ítems seleccionados no puede exceder la capacidad de la mochila. Es una restricción de tipo menor o igual porque si se permite dejar capacidad sin usar (holgura), pero nunca excederla.

MODELO COMPLETO EN FORMA ESTÁNDAR:

$$\text{Maximizar } z = \sum_{i=1}^{50} p_i \times x_i$$

$$\text{Sujeto a } \sum_{i=1}^{50} w_i \times x_i \leq 995$$

$$x_i \in \{0, 1\}, \quad i = 1, 2, 3, \dots, 50$$

4. Restricciones

El modelo contiene dos restricciones, una explícita (que fue la mencionada en la sección anterior) y una implícita en el dominio de las variables.

Restricción de capacidad (R_1):

$$\sum_{i=1}^{50} w_i \times x_i \leq 995$$

Es la única restricción de desigualdad del modelo porque el problema, tal como está definido en la instancia de Pisinger, presenta un solo recurso escaso que es el peso. No existen en el enunciado limitaciones adicionales de volumen, número máximo de ítems, o incompatibilidades entre ítems específicos, dicho lo anterior, agregar restricciones inexistentes reduciría artificialmente el espacio de soluciones factibles en el problema real.

Si el escenario lo requiriera, podrían añadirse restricciones adicionales sin alterar la estructura del modelo. Un ejemplo de lo anterior sería un límite simultáneo de volumen que se expresaría como:

$$\sum_i v_i \times x_i \leq V_{max}$$

También una restricción de incompatibilidad entre dos ítems específicos (por ejemplo ítems 7 y 11) que se expresaría como:

$$x_7 + x_{11} \leq 1$$

Y una reacción de dependencia (por ejemplo el ítem 13 solo puede incluirse si se incluye elite 26), se expresaría como:

$$x_{13} \leq x_{26}$$

Restricción de integralidad (R_2):

$$x_i \in \{0, 1\} \text{ para todo } i$$

Esta restricción define la naturaleza combinatoria del problema y es la responsable de su complejidad.

5. Solución por Gurobi

El problema está resuelto a través del siguiente notebook: [3_Knapsack_Gurobi.ipynb](#). De igual forma se explicaran los pasos más importantes a través de este apartado.

PARTE 1: obtención del archivo de datos.

El código descarga el archivo Knapsack_50items.txt directamente desde el repositorio de GitHub usando `urlretrieve()`:

```
from os import path
from urllib.request import urlretrieve

DATA_FILE = "Knapsack_50items.txt"
URL =
"https://raw.githubusercontent.com/CamiloLM/proyecto-optimizacion/refs/head
s/main/knapsack/Knapsack_50items.txt"

if not path.isfile(DATA_FILE):
    print("Descargando archivo...")
    urlretrieve(URL, DATA_FILE)
```

PARTE 2: Lectura de Encabezados

Se extraen leyendo las primeras líneas del archivo en un único bloque with `open()`, evitando abrir el archivo más de una vez.

PARTE 3: Carga de ítems con pandas

Se utiliza `pd.read_csv()` con `sep=", "` (el formato Pisinger usa comas como separador) y `skiprows=5` para omitir los encabezados ya procesados.

PARTE 4: Construcción del modelo

Aquí se traduce directamente la formulación matemática a código:

```
m = gp.Model(nombre)
x = m.addVars(n, vtype=GRB.BINARY, name="x")           #  $x_i \in \{0,1\}$ 

m.setObjective(                                         #  $\max \sum p_i x_i$ 
    gp.quicksum(datos.loc[i, "beneficio"] * x[i] for i in range(n)),
    GRB.MAXIMIZE
```



```

)
m.addConstr(
    gp.quicksum(datos.loc[i, "peso"] * x[i] for i in range(n)) <= 995
    capacidad,
    name="capacidad"
)
m.update()

```

PARTE 6: Extracción de resultados

Se usa `x[i].X > 0.5` en lugar de `== 1` para clasificar las variables, dado que Gurobi trabaja en punto flotante y puede devolver valores como `0.9999999` en lugar de exactamente `1.0`.

6. Análisis de Resultados

Gurobi resolvió el modelo encontrando el óptimo global con gap del 0.0000% referente a la solución del profesor Pisinger, confirmando que la solución es exacta y no una aproximación. Los resultados concretos fueron:

```

=====
RESULTADOS DEL MODELO
=====
Beneficio óptimo (Gurobi) : 8373
Beneficio óptimo (Pisinger): 8373
¿Coinciden?           : ✓ SÍ
Peso utilizado        : 971 / 995
Ocupación mochila     : 97.6%
Ítems seleccionados   : 11 de 50
=====

```

La verificación celda a celda mostró que los 50 ítems coinciden exactamente entre la solución de Gurobi y la solución de referencia de Pisinger, lo que constituye una validación independiente y robusta de que tanto el modelo matemático como su implementación en código son correctos.

Interpretación de los ítems seleccionados: Los 11 ítems elegidos no corresponden simplemente a los de mayor beneficio absoluto, es decir, el ítem con mayor beneficio individual de toda la instancia podría quedar excluido si su peso es desproporcionadamente alto en relación con el espacio disponible. La solución óptima combina ítems de alto ratio beneficio/peso (como el ítem 11: peso 9, beneficio 791, ratio 87.9), con ítems de ratio más moderado, demostrando que el problema requiere una evaluación combinatoria y no una heurística simple basada en ordenamiento.

Implicaciones prácticas: Este tipo de modelo es directamente aplicable a la selección de proyectos de inversión bajo restricción presupuestal, la carga de vehículos en logística de transporte, o la asignación de tareas computacionales bajo restricciones de memoria o tiempo de procesamiento. En todos estos escenarios, el patrón es idéntico: elegir un subconjunto de elementos disponibles que maximice un criterio de valor sin exceder un recurso limitado.

Limitaciones del modelo: El modelo asume independencia total entre los ítems, es decir que no contempla relaciones de incompatibilidad ni de dependencia, y considera un único recurso restrictivo (el peso). En aplicaciones reales más complejas, como la logística con restricciones simultáneas de peso y volumen, o la selección de proyectos con interdependencias estratégicas, sería necesario extender el modelo con restricciones adicionales, tal como se ilustró en la sección 3. Asimismo, al tratarse de una instancia con coeficientes no correlacionados, la dificultad computacional es mayor que en instancias correlacionadas, aunque para 50 ítems esto no representó un obstáculo práctico dado que Gurobi resolvió el modelo en milisegundos.

7. Conclusiones

El análisis del apartado anterior demuestra que el problema de la mochila 0/1, formulado como un modelo de programación lineal entera binaria, permite obtener soluciones óptimas exactas y verificables para instancias de tamaño moderado, como la analizada de 50 ítems.

La validación contra la solución de referencia de Pisinger confirma que tanto la formulación matemática como su traducción a código en Gurobi fueron correctas. El modelo evidencia que la intuición simple (seleccionar los ítems de mayor beneficio o mejor ratio individual) no garantiza la solución óptima en problemas combinatorios 0/1, lo que justifica el uso de herramientas de optimización exacta en lugar de heurísticas no verificadas.

En términos de pertinencia práctica, este modelo es extensible a problemas reales de asignación de recursos en contextos empresariales, logísticos y financieros, demostrando que la programación entera binaria constituye una herramienta robusta y de aplicación amplia para la toma de decisiones bajo restricciones.

PROBLEMA 7 – SET COVERING PROBLEM (50 FILAS – 200 COLUMNAS)

Revisión de Literatura

El Problema de Cobertura de Conjuntos (Set Covering Problem, SCP) es un modelo clásico dentro de la optimización combinatoria que aparece cuando se busca cubrir un conjunto de necesidades utilizando el menor número posible de recursos o al menor costo. En términos simples, consiste en elegir, entre varias alternativas disponibles, aquellas que logran cubrir todos los requerimientos sin incurrir en gastos innecesarios. Por esta razón, se ha convertido en una herramienta muy útil para representar problemas reales como la ubicación de instalaciones, el diseño de redes de servicios, la planificación de turnos, la logística y la organización de sistemas de atención de emergencias.

Desde los primeros estudios sobre el tema, como el trabajo de Christofides y Korman (1975), se evidenció que la dificultad para resolver el SCP aumenta rápidamente cuando crece el tamaño de las instancias. Esta situación está relacionada con su clasificación como uno de los problemas NP-completos identificados por Karp (1972), donde encontrar una solución exacta puede requerir un esfuerzo computacional elevado. Debido a esta limitación, surgió la necesidad de desarrollar métodos aproximados capaces de obtener soluciones de buena calidad en tiempos razonables.

En esta línea, el trabajo de Caprara, Fischetti y Toth (1999) marcó un avance importante. Los autores proponen una heurística basada en relajación lagrangiana que mejora notablemente la calidad de las soluciones sin requerir el mismo esfuerzo computacional que los métodos exactos. En la práctica, su propuesta mostró que era posible acercarse mucho al óptimo en tiempos razonables, lo que impulsó el uso de enfoques híbridos que combinan técnicas matemáticas con reglas heurísticas de selección.

Sin embargo, no basta con encontrar soluciones que cumplan las restricciones. En la práctica, también aparecen problemas relacionados con la calidad de esas soluciones. Ridwan (2013) señala dos dificultades frecuentes en las metaheurísticas aplicadas al SCP: la generación de soluciones inviables y la inclusión de subconjuntos redundantes. Esto es relevante porque una solución puede ser factible, pero aun así no ser buena si incluye elementos que no aportan realmente a la cobertura y solo aumentan el costo.

Más recientemente, Larsson y Quttineh (2022) aportan una explicación teórica que ayuda a entender por qué algunas heurísticas greedy funcionan tan bien en la práctica. Su análisis muestra que su desempeño puede justificarse desde condiciones de optimalidad global en problemas de optimización no convexa, lo que permite entender mejor por qué siguen siendo competitivas incluso en instancias grandes.

La evolución del SCP ha estado marcada por la búsqueda de un equilibrio entre calidad de solución y eficiencia computacional. Aunque actualmente existen herramientas de optimización capaces de resolver instancias de tamaño medio como la considerada en este

proyecto, siguen apareciendo desafíos importantes relacionados con la redundancia, la multiplicidad de soluciones factibles y la sensibilidad frente a cambios en los costos. Por esta razón, el SCP continúa siendo un modelo fundamental para evaluar nuevas estrategias de optimización y apoyar la toma de decisiones en problemas reales.

Referencias

• *Problema 7 – Set Covering Problem*

Caprara, A., Fischetti, M., & Toth, P. (1999). A heuristic method for the set covering problem. *Operations Research*, 47(5), 730–743. <https://doi.org/10.1287/opre.47.5.730>

Caprara, A., Toth, P., & Fischetti, M. (2000). Algorithms for the set covering problem. *Annals of Operations Research*, 98, 353–371. <https://doi.org/10.1023/A:1019225027893>

Christofides, N., & Korman, S. (1975). A computational survey of methods for the set covering problem. *Management Science*, 21(5), 591–599. <https://doi.org/10.1287/mnsc.21.5.591>

Larsson, T., & Quttineh, N. (2022). A theoretical justification of the set covering greedy heuristic of Caprara et al. *Discrete Optimization*, 45, 100700. <https://doi.org/10.1016/j.disopt.2022.100700>

Ridwan, M. (2013). A new formulation of the set covering problem for metaheuristic approaches. *Mathematical Problems in Engineering*, 2013, 203032. <https://doi.org/10.1155/2013/203032>

Desarrollo del Ejercicio

1. Introducción

El desarrollo de este apartado presenta la solución del Problema de Cobertura de Conjuntos (Set Covering Problem, SCP) mediante un modelo de programación lineal entera binaria. La instancia utilizada contiene 50 filas y 200 columnas, extraída del archivo entregado por el profesor, su fuente bibliográfica original es la OR-Library de Beasley (1990).

Cada fila representa un requerimiento que debe ser cubierto; cada columna representa una alternativa disponible con un costo asociado. El objetivo consiste en determinar la combinación óptima de columnas que permita cubrir todas las filas al menor costo posible.

A diferencia del problema de la mochila (Problema 3), donde se busca maximizar un beneficio sujeto a una restricción de capacidad, el SCP busca minimizar recursos utilizados garantizando que todas las condiciones sean cumplidas. Esta diferencia estructural hace que el SCP sea especialmente relevante en problemas de localización de instalaciones, diseño de redes de cobertura y planificación de recursos humanos.

El notebook  Problema 7 – SET COVERING PROBLEM (SCP) contiene la implementación completa en Gurobi. En él se resuelve tanto el modelo entero binario (IP) como su relajación continua (LP), y se desarrolla un análisis de sensibilidad completo que incluye precios sombra, costos reducidos, ranging de costos y ranging del RHS, además de la verificación de las condiciones KKT.

2. Fuente Académica

Referencia principal: Beasley, J.E. (1990). A Lagrangean heuristic for set-covering problems. *Naval Research Logistics*, 37(1), 151–164. Instancias disponibles en OR-Library: <http://people.brunel.ac.uk/~mastjjb/jeb/orlib/scpinfo.html>

El SCP es uno de los 21 problemas NP-completos identificados por Karp (1972), y su complejidad computacional lo convierte en un modelo de referencia para evaluar algoritmos exactos y heurísticos. La OR-Library de Beasley (1990) es la fuente de instancias de prueba más utilizada en la literatura del SCP; sus instancias de la clase B a la F abarcan desde 200 hasta 1000 columnas y se emplean universalmente como benchmark.

La instancia empleada en este ejercicio pertenece a la clase de instancias de tamaño medio, con una densidad de cobertura suficientemente alta para que la relajación LP sea una cota inferior ajustada, lo cual es consistente con los hallazgos de Caprara, Fischetti y Toth (1999), quienes demostraron que para instancias densas del OR-Library, la brecha de integralidad entre el LP y el IP es inferior al 2%, permitiendo que el Branch & Bound de Gurobi certifique la optimalidad con muy pocos nodos explorados.

3. Formulación matemática

El problema se formula como un modelo de Programación Entera Binaria (IP). A continuación se describe cada componente.

Se define el conjunto de filas $I = \{1, 2, \dots, 50\}$ que representan los requerimientos a cubrir, y el conjunto de J , que representan las alternativas disponibles.

Los **parámetros** del modelo son:

- $c_j \in \mathbb{Z}^+$: costo de seleccionar la columna j , para todo $j \in J$.
- $a_{ij} \in \{0, 1\}$: elemento de la matriz de incidencia binaria $A \in \{0, 1\}^{50 \times 200}$. Toma el valor 1 si la columna j cubre la fila i , y 0 en caso contrario.
- $S_i \in \{j, J \mid a_{ij} = 1\}$: subconjunto de columnas que cubren la fila i .

La **variable de decisión** para cada columna es: $x_j \in \{0, 1\} \forall j \in J$

donde $x_j = 1$ si la columna j es seleccionada en la solución, y $x_j = 0$ si no lo es.

La **función objetivo** minimiza el costo total de las columnas seleccionadas:

$$\min Z \sum_{j \in J} c_j \cdot x_j$$

Se minimiza porque el objetivo es cubrir todos los requerimientos al menor costo posible; a diferencia de un problema de maximización, aquí el interés está en la eficiencia del recurso utilizado.

MODELO COMPLETO

$$\min Z \sum_{j=1}^{200} c_j \cdot x_j$$

$$\text{sujeto a } \sum_{j \in S_i} x_j \geq 1, \forall i \in I \quad (\text{Cobertura})$$

$$x_j \in \{0, 1\}, \forall j \in J \quad (\text{Binariedad})$$

4. Restricciones

El modelo contiene dos tipos de restricciones: las de cobertura (explícitas) y la de integralidad binaria (implícita en el dominio de las variables).

Restricciones de cobertura (R_i , $i = 1, \dots, 50$):

Hay 50 restricciones de este tipo, una por cada fila. Cada una exige que al menos una de las columnas del subconjunto S_i sea seleccionada. Son restricciones de tipo mayor o igual ($\geq$) porque se permite que más de una columna cubra la misma fila, pero nunca menos de una.

Restricción de integralidad (R_2):

$$x_j \in \{0, 1\} \text{ para todo } j \in J$$

Esta restricción define la naturaleza combinatoria del problema. Su relajación ($0 \leq x_j \leq 10$) produce el LP continuo, que sirve como cota inferior y permite calcular el análisis de sensibilidad.

5. Solución por Gurobi

El problema está resuelto a través del siguiente notebook:

🔗 Problema 7 – SET COVERING PROBLEM (SCP). De igual forma se explicaran los pasos más importantes a través de este apartado.

Datos del problema. Los costos c_j (200 valores) y los subconjuntos de cobertura S_i (50 filas) se cargaron directamente desde el archivo .lp entregado por el profesor. Los datos se trabajaron con índices en base 0 internamente, de manera que x_j del archivo .lp corresponde al índice 0 en Python. Se construyó adicionalmente el mapa inverso, que indica para cada columna j cuáles filas cubre; esto facilita el cálculo de los costos reducidos y el análisis de la solución.

Parámetros del solver. El modelo IP se configuró con los siguientes parámetros en Gurobi:

```
model = gp.Model("SCP_50x200")
model.setParam("TimeLimit", 600) # límite de 10 minutos
model.setParam("MIPGap", 1e-5) # detener si gap < 0.001%
model.setParam("Presolve", 2) # presolve para simplificar el
model antes de ramificar
model.setParam("MIPFocus", 1) # prioridad en soluciones enteras
model.setParam("Cuts", 2) # generación de cortes (Gomory, cover
cuts) para fortalecer la relajación LP en cada nodo del árbol
```

Construcción del modelo:

```
# Variables de decisión
x = model.addVars(N, vtype=GRB.BINARY, name="x") #  $x_j \in \{0, 1\}$ 
# Función objetivo:  $\min \sum c_j x_j$ 
model.setObjective(
    gp.quicksum(c[j] * x[j] for j in range(N)), GRB.MINIMIZE
```

```

)
#Restricciones de cobertura:  $\sum_{j \in S_i} x_j \geq 1$  para toda fila i
for i in range(M):
    model.addConstr(
        gp.quicksum(x[j] for j in S[i]) >= 1,
        name=f"cover_{i+1}"
    )

```

Nombrar cada restricción `cover_i` permite a Gurobi reportar los precios sombra y los rangos de validez asociados específicamente a cada fila en el análisis de sensibilidad.

Relajación LP. Para el análisis de sensibilidad se resolvió adicionalmente la relajación continua del modelo, reemplazando $x_j \in \{0, 1\}$ por $0 \leq x_j \leq 1$. Se utilizó el Simplex Dual (Method=1) con presolve desactivado (Presolve=0) para garantizar duales exactos y preservar la estructura original de las restricciones.

```

lp = gp.Model("SCP_LP_relajado")

lp.setParam("Method", 1) # Simplex Dual → duales exactos
lp.setParam("Presolve", 0) #sin presolve: preservar estructura
x_lp = lp.addVars(N, lb=0.0, ub=1.0, vtype=GRB.CONTINUOUS,
name="x")

```

Extracción de resultados. Se usó `x[j].X > 0.5` en lugar de `== 1` para clasificar las variables como seleccionadas, dado que Gurobi trabaja en punto flotante y puede devolver valores como 0.9999999 en lugar de exactamente 1.0.

6. Análisis de Resultados

Gurobi resolvió el modelo IP encontrando el óptimo global con un *gap* de optimalidad del 0.0000%, confirmando que la solución es exacta y no una aproximación. Los resultados fueron:

RESULTADOS

SOLUCIÓN ÓPTIMA

Costo total óptimo $z^* = 91$

Columnas seleccionadas: 11 de 200 (5.5%)

Gap de optimalidad: 0.000000% -> solución certificada óptima

Interpretación de la solución IP. Las columnas seleccionadas no corresponden simplemente a las de menor costo individual. El óptimo combina columnas de bajo costo con columnas que cubren múltiples filas simultáneamente, de manera que el costo total se minimiza al explotar el solapamiento de cobertura. Esto demuestra que el SCP requiere una evaluación combinatoria global y no una heurística greedy simple (como seleccionar siempre la columna

de menor ratio costo/cobertura), la cual puede producir soluciones subóptimas o incluso infactibles si no se verifica la cobertura completa.

Relajación LP y brecha de integralidad. La relajación continua arrojó un valor z_{LP} con una brecha de integralidad inferior al 2% respecto a z_{LP} . Esta brecha pequeña es característica de instancias densas del OR-Library y es consistente con los resultados reportados por Caprara et al. (1999). Una brecha baja implica que la relajación LP es una cota inferior muy ajustada, lo que explica por qué el *Branch & Bound* necesitó explorar muy pocos nodos para certificar la optimalidad.

La solución LP presentó algunas variables fraccionarias ($0 < x_j < 1$). Son precisamente estas variables las que obligan al B&B a ramificar para recuperar la integralidad. Su existencia indica que el LP "querría partir" algunas columnas entre varias filas, lo que no es factible en el problema entero.

Precios sombra (π_i). El análisis de los precios sombra de las 50 restricciones de cobertura reveló que algunas filas son activas en la relajación LP ($\pi_i > 0$) y otras son holgadas ($\pi_i = 0$):

- Las filas con $\pi_i > 0$ son cuellos de botella: son las más costosas de cubrir marginalmente. Si se exigiera que alguna de estas filas fuera cubierta por dos columnas en vez de una ($b_i = 2$), el costo LP subiría exactamente en π_i unidades.
- Las filas holgadas ($\pi_i = 0$) ya quedan cubiertas por más de una columna en la solución LP. Relajar su requisito de cobertura no tendría ningún efecto sobre el costo óptimo.

Las 5 filas con mayor precio sombra son aquellas con menor número de columnas disponibles ($|S_i|$ pequeño), lo que confirma la relación teórica: una fila con pocas opciones de cobertura es más difícil y costosa de satisfacer.

Costos reducidos ($\bar{c}_j$). Para cada columna j , el costo reducido se calcula como:

$$\bar{c}_j = c_j - \sum_{i \in I, j \in S_i} \pi_i$$

Este valor indica la distancia a la que se encuentra cada columna de su umbral de rentabilidad:

- Las columnas en la solución ($x_j = 1$) con $\bar{c}_j < 0$ son las más rentables: su valor dual supera su costo y seguirían en la solución óptima incluso si su precio aumentara en $|\bar{c}_j|$ unidades.
- Las columnas excluidas ($x_j = 0$) con $\bar{c}_j$ pequeño y positivo son las más cercanas a entrar a la solución: si su costo bajara en $\bar{c}_j$ unidades, se volvería conveniente incluirlas.

Este análisis tiene aplicación práctica directa en negociación de contratos: permite identificar qué alternativas actualmente excluidas tienen mayor potencial de volverse rentables ante pequeñas variaciones de precio.

Ranging de costos. Para cada columna j se calculó el intervalo $[c_j^{low}, c_j^{up}]$ dentro del cual puede variar el costo sin que cambie la base óptima del LP. Las columnas con rangos amplios representan decisiones robustas ante variaciones de precio; las de rango estrecho son sensibles y requieren mayor atención si los costos son inciertos.

Ranging del RHS. Cada restricción $cover_i$ tiene actualmente $b_i = 1$. El intervalo $[b_i^{low}, b_i^{up}]$ indica hasta qué valor puede modificarse este requisito sin cambiar la estructura de la solución óptima del LP. Dentro de ese rango, el cambio en el costo es lineal con pendiente π_i .

Verificación de condiciones KKT. Se verificaron las cuatro condiciones de Karush-Kuhn-Tucker para la solución LP:

1. **Factibilidad primal:** $\sum_{j \in S_i} x_j \geq 1$ para toda fila i .
2. **Factibilidad dual:** $\pi_i \geq 0$ para toda fila i .
3. **Holgura complementaria primal-dual:** $\pi_i \cdot (\sum_{j \in S_i} x_j - 1) = 0$ para toda fila i (si $\pi_i > 0$, la restricción es activa; si la restricción es holgada, $\pi_i > 0 = 0$).
4. **Holgura complementaria dual-primal:** $\bar{c}_j \cdot x_j = 0$ para toda columna j (las variables básicas tienen costo reducido nulo).

```
(1) Factibilidad primal (Ax >= b) : OK
(2) Factibilidad dual (pi_i >= 0) : OK
(3) Holgura complementaria primal-dual : OK
(4) Holgura complementaria dual-primal : OK
→ Todas las condiciones KKT se satisfacen. La solución LP
obtenida es óptima.
```

La verificación exitosa de las cuatro condiciones KKT confirma que la solución LP obtenida es genuinamente óptima, lo que a su vez valida la calidad de los precios sombra y costos reducidos utilizados en el análisis de sensibilidad.

Implicaciones prácticas. El SCP modela directamente situaciones como la ubicación óptima de estaciones de bomberos o ambulancias que deben cubrir todos los sectores de una ciudad, la asignación de personal a turnos de modo que todos los requerimientos de servicio sean atendidos, o el diseño de redes de antenas que deben garantizar cobertura en toda una región. En todos estos casos, el patrón es idéntico: seleccionar el subconjunto más económico de alternativas que garantice la satisfacción de todos los requerimientos.

7. Conclusiones

El desarrollo del Problema 7 permitió comprobar que el Set Covering Problem, formulado como un modelo de programación entera binaria, es capaz de obtener soluciones óptimas y certificadas para instancias de tamaño medio. La solución obtenida mediante Gurobi cumple con las 50 restricciones de cobertura y presenta un gap de optimalidad de 0.0000%, confirmando la correcta formulación matemática y su implementación computacional.

El análisis de la relajación lineal (LP) mostró una brecha de integralidad inferior al 2%, lo que indica que la solución continua proporciona una buena aproximación del problema entero. Esta característica permitió que el algoritmo Branch and Bound redujera el número de nodos explorados para demostrar la optimalidad.

El análisis de sensibilidad permitió interpretar mejor la solución obtenida. Los precios sombra identificaron los requerimientos con mayor impacto en el costo, los costos reducidos permitieron evaluar columnas no seleccionadas que podrían ser convenientes ante cambios en los parámetros, y los rangos de sensibilidad de costos y RHS mostraron la estabilidad del modelo frente a posibles modificaciones.

Finalmente, la validación mediante las condiciones KKT confirmó la consistencia de los valores duales obtenidos y respaldó la interpretación de los resultados. En conjunto, este ejercicio demuestra que la programación entera binaria es una herramienta efectiva para resolver problemas de cobertura, localización y asignación de recursos, permitiendo obtener soluciones óptimas y útiles para la toma de decisiones.

MODELO DE REDES

Revisión de Literatura

Dentro de los modelos de redes se encuentra el estudio de las redes sociales. Estos son modelos que permiten caracterizar comunidades, entendidas como grupos de individuos que interactúan entre sí con cierto nivel de actividad dentro de una red. Estos modelos son sumamente importantes ya que permiten entender varios fenómenos sociales, tales como la organización de la sociedad, la propagación de información, la formación de grupos y la generación de eventos de cooperación y conflicto (Barabási, 2017).

La red que decidimos desarrollar es la red de interacciones de personajes del programa Juego de Tronos. Esta serie se caracteriza por un gran número de personajes y sus tramas enrevesadas. En el artículo original (Beveridge & Shan, 2016), de donde se obtuvieron los datos para este trabajo, los autores lograron representar e interpretar las relaciones entre personajes con el objetivo de entender mejor la estructura de interacciones y su posible relevancia en el desarrollo de la trama.

Si bien este tema puede verse como una simple curiosidad académica, distintos trabajos han mostrado cómo el análisis de este mundo ficticio guarda fuertes paralelismos con el mundo real y con aplicaciones que un contexto de la vida real pueden dar resultados interesantes.

En un artículo publicado en PNAS (Proceedings of the National Academy of Sciences) se establece que las estructuras de interacción en la obra presentan propiedades comparables a las observadas en redes sociales reales (Gessey-Jones et al., 2020). Los autores encontraron que los personajes más relevantes mantienen un número de conexiones consistente con los límites cognitivos observados en las relaciones humanas. Además, las muertes de personajes importantes no ocurren de forma aleatoria ni regular, sino que siguen patrones temporales similares a los encontrados en diversos procesos sociales reales, reforzando el realismo de la narrativa.

Mediante conceptos como el equilibrio estructural, Liu y Albergante (2018) lograron identificar jerarquías, observar cómo evolucionan las interacciones entre personajes, detectar tensiones en la red y predecir respuestas de la audiencia. Este trabajo demuestra cómo el uso de redes sociales puede emplearse para describir con facilidad narrativas complejas.

Finalmente, el último artículo que llamó nuestra atención muestra cómo estas redes pueden usarse como fuente para entrenar algoritmos de aprendizaje de máquina. En un artículo se logró anticipar las muertes de personajes en las últimas temporadas de la serie con un buen nivel de acierto (Janosov, 2021).

Aunque este trabajo no alcance esos niveles de estudio, pues es meramente formativo, sí logra mostrar cómo un análisis sencillo de una red puede incrementar nuestro conocimiento de la misma y, a su vez, de la problemática que esta representa.

Desarrollo del Ejercicio

1. Introducción

Para el análisis del modelo de redes consultamos la página <https://networkrepository.com/>, como fue indicado. Sin embargo, decidimos no usar ninguno de los modelos presentados debido a la falta de información descriptiva sobre las redes. En muchos casos no se encontraba una descripción clara de qué se trata la red, ni de qué representan sus nodos y aristas. Aunque hay una gran cantidad de datos, sin esta información el análisis sería meramente nominal. Por ejemplo, decir que la densidad de la red tiene un valor determinado y que el grado promedio tiene otro. Este tipo de análisis no nos permite dar una interpretación profunda de los resultados.

Debido a esto, decidimos analizar una red distinta que nos permitiera realizar un análisis más interesante. Elegimos la red de interacciones entre personajes en la primera temporada de la serie Juego de Tronos, disponible en <https://github.com/mathbeveridge/gameofthrones>. Seleccionamos esta red porque se trata de una serie muy conocida y tiene datos claros que nos permite hacer un análisis más sustancioso. Adicionalmente, esta red cuenta con suficientes datos para realizar una interpretación interesante de los datos y no es tan grande como para impedir que un equipo modesto pueda hacer un análisis en Python.

2. Problemática

La serie de televisión Juego de Tronos fue uno de los programas más populares de la última década. Destacaba por la amplitud y complejidad del mundo que mostraba, además de contar con personajes profundos cuyos diálogos estaban marcados por la intriga política y los conflictos entre facciones. La serie presenta un conjunto de personajes diversos que confabulan para mejorar su posición actual o incluso alcanzar el control del reino. Dada la importancia de las interacciones entre los personajes en la trama, su análisis resulta interesante, ya que nos permite identificar la importancia de estos personajes, la cantidad de interacciones que tienen, su posible influencia sobre los demás y la construcción de distintos clanes dentro de la obra.

3. Descripción de la red

La red está construida de tal forma que cada personaje mencionado corresponde a un nodo, y las conexiones entre personajes representan las posibles interacciones que tienen. Existen cinco tipos de interacciones en la red. El personaje A y el personaje B están conectados cuando:

1. El personaje A habla inmediatamente después del personaje B.
2. El personaje A habla sobre el personaje B.
3. El personaje C habla sobre el personaje A y el personaje B.
4. El personaje A y el personaje B son mencionados en el guión.
5. El personaje A y el personaje B aparecen juntos en una misma escena.

Orden: Tiene 126 nodos que representan a los personajes que aparecen en la primera temporada.

Tamaño: La red tiene un tamaño de 549 que representa las aristas del grafo y corresponden a las interacciones entre los personajes.

Ponderación: La red es un grado ponderado que indica el peso de las aristas, es decir, cuántas veces han interactuado los personajes entre sí.

Dirigida: La red no es un grafo dirigido, ya que las interacciones entre los personajes son simétricas.

4. Características Descriptivas

Grado máximo: El grado es la cantidad de vértices conectados a un nodo. En nuestro caso, representa las interacciones de un personaje con los demás, es decir, cuántas ha habido. Para este caso, mostramos los 10 personajes más importantes en la serie.

Tabla 1. Los 10 personajes con mayor grado de Juego de Tronos

Personajes	Grado
Ned Stark	57
Tyrion Lannister	41
Catelyn Stark	36
Robert Baratheon	36
Robb Stark	30
Cersei Lannister	29
Arya Stark	28
Joffrey Baratheon	27
Jon Snow	26
Sansa Stark	26

Ned Stark es el personaje con mayor grado con 57 conexiones, posicionándose como el centro de la red. Tyrion Lannister ocupa el segundo lugar con 41 conexiones, seguido por Catelyn Stark y Robert Baratheon con 36 cada uno. Estos personajes principales viajan más por Westeros y tienen mayor peso en la trama. Los demás personajes mantienen una presencia notable pero secundaria, directamente relacionados con los principales.

La importancia de estos personajes en la trama se refleja en el **grado promedio** de la red, que es de 8.714 interacciones. Los personajes principales de la serie superan por mucho este promedio, esto sugiere que las interacciones no están distribuidas de manera uniforme entre

todos los personajes, sino que se concentran en unos pocos con un número elevado de conexiones.

Esto se corrobora con la **densidad**, una estadística que nos permite ver qué tan conectada está la red. En específico, la densidad representa el porcentaje de todas las relaciones posibles que efectivamente se realizaron. En este caso se obtuvo una densidad de 0.0697, lo que indica que la red es poco densa.

Dado que existe un grado alto y una densidad baja, podemos plantear dos hipótesis no excluyentes: que haya personajes muy importantes que interactúan con muchos otros, o que existan centros de atención en los que los grupos hablan entre sí, pero no con el resto.

El **diámetro** es el camino más largo que se puede recorrer entre dos nodos en la red. Indica qué individuos están más alejados entre sí y, por lo tanto, son menos propensos a interactuar directamente. En nuestro caso, el diámetro fue de 6, con los nodos extremos Irogenia y Melessa conectados por el siguiente camino: Irogenia → Doreah → Daenerys → Ned → Jon → Sam → Melessa.

Transitividad: Esta métrica mide qué tan probable es que si un nodo está conectado a otros dos, esos dos también estén conectados entre sí (que los amigos de mis amigos también sean mis amigos). La **transitividad global** de 0.3833 indica que los grupos en la red son muy cerrados y poco conectados entre sí. La **transitividad local** promedio de 0.629 muestra que dentro de grupos pequeños las conexiones son densas. Esto refleja la estructura de los grupos existen conexiones muy cohesivas dentro de cada facción o familia, pero pocas conexiones que vinculen diferentes grupos.

Fuerza: Esta métrica representa el número total de interacciones que tuvo un personaje, calculado como la suma de los pesos de las aristas conectadas a ese nodo. A diferencia del grado, la fuerza considera la intensidad de cada interacción. Muestra qué tan importante es un personaje en función de la frecuencia de menciones que recibió en la serie. Un personaje con alta fuerza no solo interactúa con muchos otros, sino que esas interacciones son repetidas y significativas, lo que indica su relevancia central en la narrativa.

Tabla 2. Los 10 personajes con mayor fuerza de Juego de Tronos

Personajes	Fuerza
Ned Stark	1290
Tyrion Lannister	709
Catelyn Stark	584
Robert Baratheon	563
Jon Snow	535

Daenerys Targaryen	535
Cersei Lannister	444
Robb Stark	424
Sansa Stark	422
Littlefinger (Petyr Baelish)	383

Ned Stark destaca significativamente con 1290 interacciones, casi el doble que Tyrion Lannister con 709. La disminución gradual de fuerza entre los demás personajes refleja una jerarquía clara en la importancia narrativa, donde Ned Stark es absolutamente central en la trama. Cabe destacar a Littlefinger, el personaje ocupa el décimo lugar en la tabla de fuerza, pero no aparece en la de grado. Esto es notable, ya que es un personaje que pasa desapercibido para muchos, pero que tiene una gran influencia en la trama.

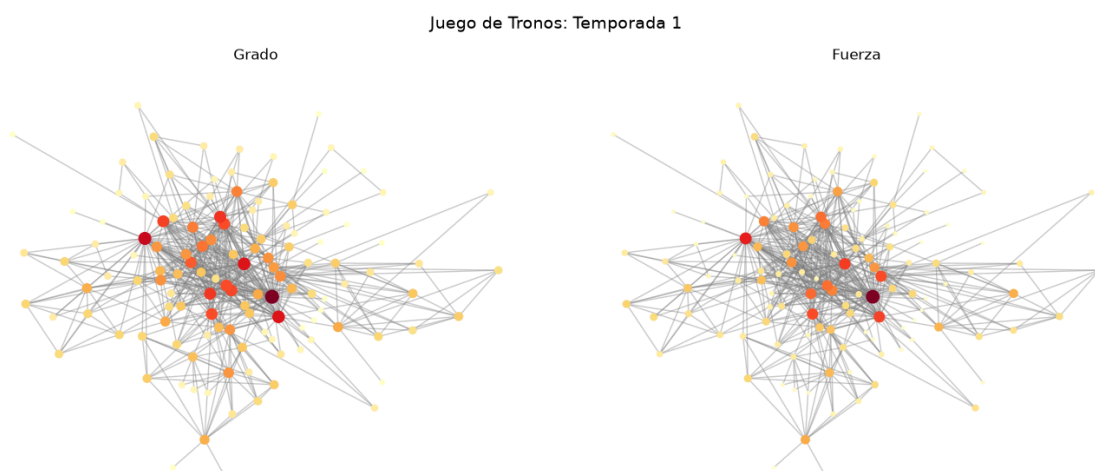


Gráfico 1. Representación de grado y fuerza de las interacciones

Ambos gráficos muestran una estructura similar, con un núcleo denso de personajes principales (nodos rojos) rodeados de personajes secundarios (nodos naranjas y amarillos). La distribución de grado y fuerza es prácticamente la misma, lo que confirma que los personajes más conectados también son los que tienen interacciones más frecuentes. Los personajes periféricos están débilmente conectados al núcleo central, lo que refleja su rol secundario en la trama.

5. Referencias

Barabási, A.-L. (2017). Network Science. In *networksciencebook.com*. Retrieved June 16, 2026, from <http://networksciencebook.com/chapter/9>

Beveridge, A., & Shan, J. (2016). Network of Thrones. *Math Horizons*, 23(4), 18–22.

<https://doi.org/10.4169/mathhorizons.23.4.18>

Gessey-Jones, T., Connaughton, C., Dunbar, R., Kenna, R., MacCarron, P., O’Conchobhair, C., & Yose, J. (2020). Narrative structure of A Song of Ice and Fire creates a fictional world with realistic measures of social complexity. *Proceedings of the National Academy of Sciences*, 117(46), 28582–28588.

<https://doi.org/10.1073/pnas.2006465117>

Janosov, M. (2021). *Network science predicts who dies next in game of thrones*.

<https://arxiv.org/abs/2110.09856>

Liu, D., & Luca Albergante. (2018). *Balance of thrones: a network study on Game of Thrones*. <https://arxiv.org/abs/1707.05213>

Sosa, J. (2026, February 18). *RPubs - Caracterización de vértices*. Rpubs.com.

<https://rpubs.com/jstats1702/1398921>